



**Departamento de Engenharia de Informática e de  
Computadores**

## **Relatório do projeto**

### **Gestão de mercados financeiros — Financial market management**

Autores:    35164        Fernando Duarte  
                 52197        Gabriel Ferreira

Relatório para a Unidade Curricular de Sistemas de informação  
da Licenciatura em Engenharia Informática e de Computadores

Professores: Ana Beire, João Vitorino, Matilde Pato e Nuno Datia

06- 06 – 2026

## Resumo

O objectivo deste documento é apresentar um relatório sobre o projeto de Gestão de mercados financeiros, tendo como maior foco a implementação da lógica de negócio, recorrendo ao que foi leccionado no decorrer do semestre tendo como base as seguintes temáticas:

- Processamento Transaccional;
- Bases de dados Dinâmicas;
- Indexação e planos de execução;
- Camada de acesso a dados.

Pode-se observar no trabalho a implementação das temáticas mencionadas e iremos abordar o uso das técnicas e a com justificação das mesmas, seja de forma explicita ou implicita. Focou-se a implementação numa abordagem híbrida que combina lógica de base de dados ativa (gatilhos, funções e procedimentos armazenados em PostgreSQL) com uma camada de acesso a dados em Java, recorrendo à especificação Jakarta Persistence (JPA). Os resultados validam a aplicação de padrões de desenho, controlo de concorrência com "optimistic locking" e a correta gestão e libertação de recursos.

## Índice

|                                                |    |
|------------------------------------------------|----|
| 1. Introdução .....                            | 4  |
| 1.1 Restrições de integridade .....            | 5  |
| 2. Resultados .....                            | 5  |
| 2.1 Restrições de integridade .....            | 6  |
| 2.2 Função fx-media-movel .....                | 8  |
| 2.3 Função fx_portefolio_info .....            | 9  |
| 2.4 Procedimento p_actualizaValorDiario .....  | 10 |
| 2.5 Vista contacto_cliente .....               | 14 |
| 2.7 Aplicação Java .....                       | 19 |
| 2.8 Como Compilar e Executar a Aplicação ..... | 21 |
| 2.9 Teste Optimistic Locking .....             | 21 |
| 3. Conclusão .....                             | 22 |
| Referências .....                              | 23 |

## Listagens

|                                                      |    |
|------------------------------------------------------|----|
| Listagem 1 Implementação validate NIF .....          | 2  |
| Listagem 2 Implementação check duplicated .....      | 4  |
| Listagem 3 Implementação check media movel.....      | 5  |
| Listagem 4 Implementação portfolio info .....        | 6  |
| Listagem 5 Implementação atualiza valor diario ..... | 10 |
| Listagem 6 Implementação view contacto_cliente.....  | 13 |
| Listagem 7 Implementação update market value.....    | 15 |

## 1. Introdução

O presente trabalho enquadra-se no desenvolvimento de um sistema de informação para a empresa ePaus.

Tendo como base os resultados pretendidos no trabalho, iremos abordar cada questão pela ordem numérica que está apresentada no enunciado utilizando o sistema de base de dados que nos foi providenciado, melhorando e aplicando as técnicas aprendidas ao longo do semestre.

### 1.1 Ambiente de Desenvolvimento e Ferramentas

Para a elaboração e validação deste projeto, foi essencial o uso de um conjunto de ferramentas que permitiram a ligação entre o desenvolvimento local e a infraestrutura da instituição:

**Acesso à Base de Dados (PostgreSQL):** A base de dados principal encontra-se alojada nos servidores do ISEL. Para aceder de forma segura a este ambiente, foi utilizada a VPN disponibilizada, o que permitiu estabelecer uma ligação com a base de dados remota a partir das nossas máquinas locais.

**Gestão de Base de Dados (DBeaver):** Utilizamos o DBeaver como interface gráfica principal para interagir com o PostgreSQL. Esta ferramenta foi crucial em três fases:

1. Para correr os "scripts" iniciais de criação do modelo físico (``01-create-model.sql``) e introdução de dados base (``02-insert-data.sql``).
2. Para implementar e compilar o nosso código SQL ativo (``03-resolution.sql``), incluindo as funções, procedimentos e gatilhos.
3. Para monitorizar e verificar o estado da base de dados em tempo real, validando se as alterações feitas pela nossa aplicação Java (como a inserção de novos clientes ou atualização de valores) se refletiam corretamente nas tabelas e vistas, garantindo a comunicação correta do sistema.

**Gestão do Projeto Java (Apache Maven):** A estrutura do projeto Java foi gerida e construída com recurso ao Maven. Esta ferramenta permitiu-nos gerir dependências (como os "drivers" do PostgreSQL e as bibliotecas do JPA), compilar os múltiplos módulos (``dal``, ``model`` e ``app``) e executar a aplicação final de forma padronizada através da linha de comandos.

## 2. Resultados

### 2.1 Restrições de integridade

Para validar um NIF português temos de verificar se tem exactamente 9 dígitos.

De forma a garantir a restrição, procedemos ao uso de operadores e expressões em Regex, no qual utiliza-se o operador “!~” para validar a expressão “**^[0-9]{9}\$**”. Os elementos ‘^’ e ‘\$’ representam o início e o fim da string, respetivamente, o elemento ‘[0-9]’ verifica se só existem dígitos entre 0 e 9 e o elemento ‘{9}’ verifica se existem 9 dígitos.

O ‘**RAISE EXCEPTION**’ permite-nos interromper a inserção ou atualização na tabela cliente mantendo a integridade da mesma, o que garante o objetivo deste trigger. Poderia-se utilizar o trigger ‘**AFTER**’ neste caso porque temos a validação na mesma, o que permite o rollback da inserção ou atualização, contudo não é usual utilizar AFTER em contexto de inserção ou atualização de dados.

```
CREATE OR REPLACE FUNCTION validate_NIF_func() RETURNS TRIGGER LANGUAGE
plpgsql AS $$
BEGIN
    IF NEW.nif IS NULL OR NEW.nif !~ '^[0-9]{9}$' THEN
        RAISE EXCEPTION 'NIF inválido! Tem de conter exatamente 9 dígitos
numéricos.';
    END IF;
    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER validate_NIF
    BEFORE INSERT OR UPDATE
    ON cliente
    FOR EACH ROW
EXECUTE FUNCTION validate_NIF_func();
```

Listagem 1 Implementação validate NIF

Este trigger tem como objetivo verificar duplicados de email e telefone na inserção e atualização das tabelas de contacto\_email e contacto\_telefone.

Procedeu-se à implementação de uma só função para a validação dos duplicados com recurso a variáveis especiais do PostgreSQL **"TG\_OP"** e **"TG\_TABLE\_NAME"**, as quais permitem identificar o tipo de operação e tabela aquando a ativação do trigger e assim utilizando uma só função para validar os duplicados. Utilizou-se o **"RAISE EXCEPTION"** para parar a inserção ou atualização caso seja detetado um duplicado.

```
CREATE OR REPLACE TRIGGER check_duplicated_email_trg
  BEFORE INSERT OR UPDATE OF email, cliente_nif
  ON contacto_email
  FOR EACH ROW
EXECUTE FUNCTION check_duplicated_contact_func();

CREATE OR REPLACE TRIGGER check_duplicated_telefone_trg
  BEFORE INSERT OR UPDATE OF telefone, cliente_nif
  ON contacto_telefone
  FOR EACH ROW
EXECUTE FUNCTION check_duplicated_contact_func();
```

```

CREATE OR REPLACE FUNCTION check_duplicated_contact_func()
    RETURNS TRIGGER
    LANGUAGE plpgsql
AS
$$
BEGIN
    -- Verificar duplicado para a tabela contacto_email
    IF TG_TABLE_NAME = 'contacto_email' THEN
        IF (TG_OP = 'INSERT') OR (TG_OP = 'UPDATE' AND NEW.email IS DISTINCT
FROM OLD.email) THEN
            IF EXISTS(
                SELECT 1
                FROM contacto_email
                WHERE cliente_nif = NEW.cliente_nif
                    AND email = NEW.email
            ) THEN
                RAISE EXCEPTION 'Email % já existe para este cliente!',
NEW.email;
            END IF;
        END IF;
    END IF;

    -- Verificar duplicado para a tabela contacto_telefone
    IF TG_TABLE_NAME = 'contacto_telefone' THEN
        IF (TG_OP = 'INSERT') OR (TG_OP = 'UPDATE' AND NEW.telefone IS
DISTINCT FROM OLD.telefone) THEN
            IF EXISTS(
                SELECT 1
                FROM contacto_telefone
                WHERE cliente_nif = NEW.cliente_nif
                    AND telefone = NEW.telefone
            ) THEN
                RAISE EXCEPTION 'Telefone % já existe para este cliente!',
NEW.telefone;
            END IF;
        END IF;
    END IF;

    RETURN NEW;
END;
$$;

```

Listagem 2 Implementação check duplicated



## 2.2 Função fx-media-movel

Esta função foi implementada para calcular a média móvel dos últimos N dias. Utilizou-se uma sub-consulta (subquery) para extrair os valores de fecho do instrumento selecionado ordenados de forma decrescente pela data, limitados ao parâmetro indicado. Sobre este conjunto de dados, aplicou-se a função de agregação "AVG", retornando o resultado.

```
CREATE OR REPLACE FUNCTION fx_media_movel(days INTEGER, instrumento_isin
VARCHAR(12))
    RETURNS NUMERIC
    LANGUAGE plpgsql
AS
$$
DECLARE
    media NUMERIC;
BEGIN
    -- Calcula a média móvel dos últimos N dias a partir dos registos diários
do instrumento
    SELECT AVG(sub.valor_fecho)
    INTO media
    FROM (SELECT vid.valor_fecho
          FROM valor_instrumento_diario vid
          WHERE vid.instrumento_isin = fx_media_movel.instrumento_isin
          ORDER BY vid.data DESC
          LIMIT days) sub;

    RETURN media;
END;
$$;
```

Listagem 3 Implementação check media movel

## 2.3 Função fx\_portefolio\_info

Esta função tem como objetivo a produção de uma listagem de portfólio incluindo o ISIN, a quantidade, o valor actual, e a percentagem de variação. Optou-se pelo retorno de uma tabela com os campos mencionados, para tal, com recurso ao identificador do portefólio é possível os dados pedidos com um junção das tabelas posicao e dados\_fundamentais as quais contém ISIN do instrumento, a quantidade, valor actual e percentagem de variação.

É possível utilizar o **"RETURN QUERY"** porque o resultado da função corresponde diretamente ao resultado de uma consulta **"SELECT"**, e assim é possível preencher as colunas declaradas na função.

NOTA: Seria possível utilizar a view **"valor\_posicao"** se adicionarmos a coluna "variacao\_de\_percentagem\_diária" da tabela "dados\_fundamentais". Com esta opção evitaríamos a utilização da junção de tabelas e apenas necessitaríamos de executar a query à view.

```
CREATE OR REPLACE FUNCTION fx_portefolio_info(p_portefolio_id BIGINT)
  RETURNS TABLE
  (
    isin                VARCHAR(12),
    quantidade          NUMERIC,
    valor_actual        NUMERIC,
    percentagem_variacao_diaria NUMERIC
  )
  LANGUAGE plpgsql
AS
$$
BEGIN
  -- Devolve uma listagem com os dados requeridos, cruzando posições com
  dados fundamentais
  RETURN QUERY
    SELECT pos.instrumento_isin,
           pos.quantidade,
           df.valor_actual,
           df.percentagem_variacao_diaria
    FROM posicao AS pos
         JOIN
           dados_fundamentais AS df ON pos.instrumento_isin =
df.instrumento_isin
    WHERE pos.portefolio = p_portefolio_id;
END;
$;
```

Listagem 4 Implementação portfolio info

## 2.4 Procedimento p\_atualizaValorDiario

Este procedimento armazenado atualiza os valores diários dos instrumentos financeiros processando dados em massa ("batch"). Através de um iterador ("FOR LOOP") sobre os registos distintos inseridos na tabela "triplo\_externo" com a data corrente, extraem-se os valores mínimo, máximo, de abertura (primeiro registo do dia) e de fecho (último registo do dia). De modo a acautelar o recálculo e preservar a robustez, os dados são inseridos na tabela "valor\_instrumento\_diario" com recurso à cláusula "ON CONFLICT DO UPDATE", garantindo uma operação idempotente que substitui os dados caso já existam para esse dia.

```
CREATE OR REPLACE PROCEDURE p_atualizaValorDiario()
    LANGUAGE plpgsql
AS
$$
DECLARE
    instrumento_rec RECORD;
    min_val          NUMERIC;
    max_val          NUMERIC;
    close_val        NUMERIC;
    open_val         NUMERIC;
    data_atual DATE := CURRENT_DATE;
    previous_close NUMERIC;
    six_month_close NUMERIC;
BEGIN
    -- Loop por cada instrumento que teve triplos hoje
    FOR instrumento_rec IN
        SELECT DISTINCT te.identificador
        FROM triplo_externo te
        JOIN instrumento i
            ON i.instrumento_id = te.identificador
        WHERE data_tempo::DATE = data_atual
    LOOP
        -- Descobrir o valor min e max de hoje
        SELECT MIN(te.valor), MAX(te.valor)
        INTO min_val, max_val
        FROM triplo_externo te
        WHERE te.identificador = instrumento_rec.identificador
            AND te.data_tempo::DATE = data_atual;
```

```

-- Descobrir o valor de fecho
SELECT te.valor
INTO close_val
FROM triplo_externo te
WHERE te.identificador = instrumento_rec.identificador
      AND te.data_tempo::DATE = data_atual
ORDER BY te.data_tempo DESC
LIMIT 1;

-- Descobrir o valor de abertura
SELECT te.valor
INTO open_val
FROM triplo_externo te
WHERE te.identificador = instrumento_rec.identificador
      AND te.data_tempo::DATE = data_atual
ORDER BY data_tempo ASC
LIMIT 1;

-- Guardar tudo na tabela valor_instrumento_diario (Inserir ou Atualizar se
já existir conflito em data)
      INSERT INTO valor_instrumento_diario (
          instrumento_isin,
          data,
          valor_minimo,
          valor_maximo,
          valor_abertura,
          valor_fecho)
      VALUES (
instrumento_rec.identificador,
          data_atual,
min_val,
max_val,
open_val,
close_val)
      ON CONFLICT (instrumento_isin, data) DO UPDATE
      SET valor_minimo = EXCLUDED.valor_minimo,
          valor_maximo = EXCLUDED.valor_maximo,
          valor_abertura = EXCLUDED.valor_abertura,
          valor_fecho = EXCLUDED.valor_fecho;

```

```

/*
    Valor de fecho mais próximo de há seis meses.
    Usado para calcular a variação a seis meses.
*/
SELECT vid.valor_fecho
INTO six_month_close
FROM valor_instrumento_diario vid
WHERE vid.instrumento_isin = instrumento_rec.identificador

```

```

    AND vid.data <= data_atual - 180
ORDER BY vid.data DESC
LIMIT 1;

```

```

-- Guardar tudo na tabela dados_fundamentais(Inserir ou Atualizar se já
existir conflito em data)
    INSERT INTO dados_fundamentais (
        instrumento_isin,
        variacao_diaria,
        valor_atual,
        media_6_meses,
        variacao_6_meses,
        percentagem_variacao_diaria,
        percentagem_variacao_6_meses
    )
    VALUES (
        instrumento_rec.identificador,
        ROUND(max_val - min_val, 2),
        ROUND(close_val, 2),
        ROUND(COALESCE(fx_media_movel(180,
instrumento_rec.identificador),close_val),2),
        ROUND(close_val - COALESCE(six_month_close,
close_val),2),
        CASE
            WHEN previous_close IS NULL OR previous_close = 0
THEN 0
            ELSE ROUND(((close_val - previous_close) /
previous_close) * 100, 2)
        END,
        CASE
            WHEN six_month_close IS NULL OR six_month_close =
0 THEN 0
            ELSE ROUND(((close_val - six_month_close) /
six_month_close) * 100, 2)
        END
    )
    ON CONFLICT (instrumento_isin) DO UPDATE
        SET variacao_diaria = EXCLUDED.variacao_diaria,
            valor_atual = EXCLUDED.valor_atual,
            media_6_meses = EXCLUDED.media_6_meses,
            variacao_6_meses = EXCLUDED.variacao_6_meses,
            percentagem_variacao_diaria =
EXCLUDED.percentagem_variacao_diaria,
            percentagem_variacao_6_meses =
EXCLUDED.percentagem_variacao_6_meses;

    END LOOP;
END;
$$;

```

Listagem 5 Implementação atualiza valor diario

## 2.5 Vista contacto\_cliente

A vista "contacto\_cliente" foi concebida para agregar e apresentar, num único ponto de acesso, a informação transversal dos clientes e dos seus respetivos contactos. Foi construída com recurso a um "UNION ALL" entre as junções da tabela cliente com as de contactos (email e telefone).

Por ser uma vista complexa, a sua manipulação não é nativamente suportada. Para suprimir essa limitação, desenhou-se um gatilho do tipo "INSTEAD OF" (trg\_contacto\_cliente\_dml) que interceta operações de Inserção, Atualização e Eliminação (DML). Este delega fisicamente a gravação da informação na tabela "cliente" e na tabela de contactos respetiva, cumprindo de forma transparente a separação e coerência do modelo relacional.

```
CREATE OR REPLACE VIEW contacto_cliente(nif, cartao_cidadao, nome,
tipo_contacto, contacto, descricao)
AS
SELECT c.nif,
       c.cartao_cidadao,
       c.nome,
       'email' AS tipo_contacto,
       ce.email AS contacto,
       ce.descricao
FROM cliente c
      JOIN
      contacto_email ce ON c.nif = ce.cliente_nif
UNION ALL
SELECT c.nif,
       c.cartao_cidadao,
       c.nome,
       'telefone' AS tipo_contacto,
       ct.telefone AS contacto,
       ct.descricao
FROM cliente c
      JOIN
      contacto_telefone ct ON c.nif = ct.cliente_nif;
```

```

CREATE OR REPLACE FUNCTION fn_contacto_cliente_dml()
    RETURNS TRIGGER
    LANGUAGE plpgsql
AS
$$
BEGIN
    -- INSERT
    IF TG_OP = 'INSERT' THEN
        -- Verifica se o cliente já existe, se não, cria-o
        IF NOT EXISTS(SELECT 1 FROM cliente WHERE nif = NEW.nif) THEN
            INSERT INTO cliente(nif, cartao_cidadao, nome)
            VALUES (NEW.nif, NEW.cartao_cidadao, NEW.nome);
        END IF;

        -- Insere o contacto na tabela correspondente
        IF NEW.tipo_contacto = 'email' THEN
            INSERT INTO contacto_email(cliente_nif, email, descricao)
            VALUES (NEW.nif, NEW.contacto, NEW.descricao);
        ELSIF NEW.tipo_contacto = 'telefone' THEN
            INSERT INTO contacto_telefone(cliente_nif, telefone, descricao)
            VALUES (NEW.nif, NEW.contacto, NEW.descricao);
        ELSE
            RAISE EXCEPTION 'Tipo de contacto inválido: %',
NEW.tipo_contacto;
        END IF;

        RETURN NEW;
    -- UPDATE
    ELSIF TG_OP = 'UPDATE' THEN
        -- Atualiza os dados do cliente
        UPDATE cliente
        SET nome = NEW.nome,
            cartao_cidadao = NEW.cartao_cidadao
        WHERE nif = OLD.nif;

        -- Atualiza o contacto na tabela correspondente
        IF OLD.tipo_contacto = 'email' THEN
            UPDATE contacto_email
            SET email = NEW.contacto,
                descricao = NEW.descricao
            WHERE cliente_nif = OLD.nif
                AND email = OLD.contacto;
        ELSIF OLD.tipo_contacto = 'telefone' THEN
            UPDATE contacto_telefone
            SET telefone = NEW.contacto,
                descricao = NEW.descricao
            WHERE cliente_nif = OLD.nif
                AND telefone = OLD.contacto;
        END IF;
    END IF;

```



```
RETURN NEW;
```

```
-- DELETE
ELSIF TG_OP = 'DELETE' THEN
    -- Remove o contacto na tabela correspondente
    IF OLD.tipo_contacto = 'email' THEN
        DELETE
        FROM contacto_email
        WHERE cliente_nif = OLD.nif
            AND email = OLD.contacto;
    ELSEIF OLD.tipo_contacto = 'telefone' THEN
        DELETE
        FROM contacto_telefone
        WHERE cliente_nif = OLD.nif
            AND telefone = OLD.contacto;
    END IF;

    RETURN OLD;
END IF;
END;
$$;

CREATE OR REPLACE TRIGGER trg_contacto_cliente_dml
    INSTEAD OF INSERT OR UPDATE OR DELETE
    ON contacto_cliente
    FOR EACH ROW
EXECUTE FUNCTION fn_contacto_cliente_dml();
```

Listagem 6 Implementação view contacto\_cliente

## 2.6 Lógica de Negócio Adicional na Base de Dados

Para além dos requisitos explícitos, foi implementada uma regra de negócio adicional para garantir a coerência dos dados a nível do mercado. A função `update\_market\_value\_func()`, ativada pelo gatilho `update\_market\_value\_trg` sempre que um valor diário de um instrumento é inserido ou atualizado, recalcula e atualiza o valor total do mercado a que esse instrumento pertence.

```
CREATE OR REPLACE FUNCTION update_market_value_func()
    RETURNS TRIGGER
    LANGUAGE plpgsql
AS
$$
DECLARE
    v_mercado          VARCHAR(20);
    v_sum_abertura     NUMERIC;
    v_ontem            DATE;
    v_valor_abertura_mercado NUMERIC;
BEGIN
    -- Obter o mercado associado ao instrumento modificado
    SELECT mercado INTO v_mercado FROM instrumento WHERE instrumento_id =
NEW.instrumento_isin;

    -- Calcular a nova soma dos valores de abertura de todos os instrumentos
desse mercado para o dia em causa
    SELECT SUM(valor_abertura)
    INTO v_sum_abertura
    FROM valor_instrumento_diario vid
        JOIN instrumento i ON i.instrumento_id = vid.instrumento_isin
    WHERE i.mercado = v_mercado
        AND vid.data = NEW.data;
```

```

-- Obter o valor de índice do mercado do dia anterior
v_ontem := NEW.data - 1;
SELECT valor_indice
INTO v_valor_abertura_mercado
FROM valor_mercado_diario
WHERE mercado = v_mercado
      AND data = v_ontem;

IF v_valor_abertura_mercado IS NULL THEN
    v_valor_abertura_mercado := 0;
END IF;

-- Atualiza o valor diário do mercado
INSERT INTO valor_mercado_diario(mercado, data, valor_indice,
valor_abertura, variacao_diaria)
VALUES (v_mercado, NEW.data, v_sum_abertura, v_valor_abertura_mercado,
        v_sum_abertura - v_valor_abertura_mercado)
ON CONFLICT (mercado, data) DO UPDATE
    SET valor_indice = EXCLUDED.valor_indice,
        valor_abertura = EXCLUDED.valor_abertura,
        variacao_diaria = EXCLUDED.variacao_diaria;

RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER update_market_value_trg
AFTER INSERT OR UPDATE
ON valor_instrumento_diario
FOR EACH ROW
EXECUTE FUNCTION update_market_value_func();

```

Listagem 7 Implementação update market value

## 2.7 Aplicação Java

A base da nossa aplicação Java reside no mapeamento objeto-relacional das tabelas e vistas da base de dados para classes de entidade. Para este projeto, criámos as seguintes entidades principais:

`Cliente.java`: A entidade central que representa os dados de um cliente. Nela, incluímos também o atributo `version` anotado com `@Version`, que é a peça fundamental para o controlo de concorrência com optimistic locking.

`Portefolio.java`: Representa um portfólio de um cliente, estabelecendo a relação `@ManyToOne` com a entidade `Cliente`, o que nos permite navegar e consultar facilmente os portfólios associados a um cliente.

`ContactoCliente.java`: Esta é uma entidade diferente, pois não mapeia uma tabela, mas sim a `VIEW` `contacto\_cliente`. O seu propósito é servir como um objeto para a persistência de dados, permitindo-nos usar `em.persist()` de forma simples e delegar a lógica de inserção para o `trigger` da base de dados.

A nossa camada de acesso a dados foi desenvolvida com JPA, tirando partido de vários padrões de desenho que o `EntityManager` implementa para nós através da manipulação destas entidades.

Criar um cliente e respetivo contacto: Para esta funcionalidade, aplicámos o padrão Data Mapper. Em vez de interagirmos diretamente com a base de dados, criámos uma entidade, `ContactoCliente.java`, que mapeia a `VIEW` com o mesmo nome. A utilização deste padrão foi feita pelo `EntityManager` que, ao invocarmos `em.persist(ccObj)`, atuou como o "mapeador", convertendo o nosso objeto Java num comando `INSERT` sobre a `VIEW`. Esta abordagem permitiu-nos delegar a lógica de negócio complexa (a separação do cliente e do contacto) para o `trigger` da base de dados, mantendo o nosso código Java limpo e focado na persistência do objeto.

Criar um portfólio de um cliente: Neste método, o `EntityManager` aplicou por nós o padrão Identity Map. Ao utilizarmos `em.find(Cliente.class, nif)`, o `EntityManager` primeiro verifica no seu mapa de identidade interno se já existe uma instância do cliente com aquele NIF em memória. Se existir, ele devolve-a imediatamente, evitando uma ida desnecessária à base de dados. Caso contrário, ele busca o cliente, guarda-o no seu mapa para futuras utilizações e devolve-o. Adicionalmente, configurámos a relação entre `Portefolio` e `Cliente` para usar Lazy Loading. Este padrão de performance instrui o JPA a não carregar os dados completos do cliente ao buscar um portfólio, carregando apenas a sua referência (ID), o que otimiza o desempenho.

Listar as posições de um cliente: Aqui, utilizámos o padrão Query Object. Em vez de concatenarmos uma `String` SQL, construímos uma consulta segura e orientada a objetos com `em.createQuery("SELECT p FROM Portefolio p ...")`. Este método devolve um objeto `Query` que representa a nossa intenção de busca. A grande vantagem é que o JPA valida a sintaxe JPQL e trata do "binding" de parâmetros de forma segura, prevenindo ataques de SQL Injection.

Atualização dos dados de um cliente: Esta funcionalidade demonstra o padrão Unit of Work. Após carregarmos o objeto ``cliente`` com ``em.find()``, ele passa a ser gerido pelo ``EntityManager``. Quando modificamos os seus dados através dos ``setters`` (e.g., ``cliente.setNome(newName)``), a "unidade de trabalho" do ``EntityManager`` regista essa alteração. No momento do ``tx.commit()``, ela analisa todos os objetos que gere, deteta que o ``cliente`` foi modificado e gera automaticamente o comando ``UPDATE`` necessário, sem que precisemos de o escrever manualmente.

## 2.8 Como Compilar e Executar a Aplicação

O projeto foi configurado para ser gerido com o Apache Maven. Para compilar e executar a aplicação a partir da linha de comandos, são necessários os seguintes passos, executados a partir da diretoria raiz do projeto:

### 1. Compilar e Empacotar o Projeto:

Este comando compila todos os módulos (``model``, ``dal`` e ``app``), executa os testes e empacota o resultado.

```
mvn clean install
```

### 2. Executar a Aplicação:

Após a compilação bem-sucedida, este comando executa a classe principal da aplicação.

```
mvn -pl app exec:java
```

## 2.9 Teste Optimistic Locking

Para garantir a integridade dos dados em cenários de acessos concorrentes, implementámos o padrão de controlo de concorrência **\*\*Optimistic Locking\*\***. Para isso, adicionámos um atributo `version` anotado com `@Version` na nossa entidade `Cliente`. O JPA usa este campo para verificar se um registo foi alterado por outro processo entre o momento da sua leitura e da sua escrita.

O nosso teste para validar este mecanismo seguiu os seguintes passos:

1. Iniciámos a operação de atualização de um cliente na nossa aplicação Java. Neste momento, o `EntityManager`, a nossa Unit of Work, carregou o cliente para a sua memória (para o seu Identity Map) com uma versão específica, digamos, a versão X.
2. Com a aplicação Java em pausa (à espera de `input`), simulámos um acesso concorrente. Utilizando o DBeaver, executámos um comando `UPDATE` diretamente na tabela `cliente`, alterando o nome do mesmo registo. Esta ação fez com que o PostgreSQL incrementasse o valor da coluna `version` para Y.
3. Retomámos a aplicação Java, inserimos os novos dados e mandámos a transação continuar com `tx.commit()`.
4. Ao fazer o `commit`, o `EntityManager` tentou executar o seu `UPDATE`, mas adicionou uma condição: `WHERE version = X`. Como a versão na base de dados já era Y, a condição falhou e o `UPDATE` não afetou nenhuma linha.
5. O JPA detetou que a sua operação de escrita não teve o efeito esperado e, por isso, lançou uma `OptimisticLockException`.
6. Capturámos esta exceção específica num bloco `catch`, abortámos a transação com `tx.rollback()` e apresentámos uma mensagem de erro clara ao utilizador, prevenindo uma "lost update".

### **3. Conclusão**

Com a conclusão deste projeto, conseguimos implementar com sucesso todas as funcionalidades propostas para o sistema da empresa ePaus. A divisão das responsabilidades mostrou-se bastante eficaz: deixámos as regras críticas de integridade como a validação do NIF ou a garantia de que não há contactos repetidos diretamente ao encargo da base de dados, utilizando funções e gatilhos. Por outro lado, a aplicação Java encarregou-se da interação transacional e do mapeamento dos dados.

A utilização do JPA simplificou de forma notória a nossa camada de acesso a dados. Ao compreendermos e aplicarmos os padrões de desenho que o JPA utiliza, constatámos que grande parte do código SQL manual pode ser evitado, tornando o desenvolvimento mais ágil e seguro.

Em suma, este trabalho permitiu consolidar os conceitos teóricos da unidade curricular.

## Referências

- [1] PostgreSQL Global Development Group. (2024). *PostgreSQL 18 Documentation*. Disponível em: [PostgreSQL: Documentation: 18: PostgreSQL 18.4 Documentation](#).
- [2] Eclipse Foundation. (2022). *Jakarta Persistence 3.1 Specification*. Disponível em: <https://jakarta.ee/specifications/persistence/3.1/>.